Symbiosis Institute of Technology, Nagpur

Symbiosis International (Deemed University)

Established under Section 3 of the UGC Act, 1956 vide notification No. F.9-12/2001-U.3 of Government of India

Re-accredited by NAAC with 'A++' grade

Experiment No.: 11 Name: Vansh Mahant PRN: 24070521038 Section: A Batch: A1

1. Title

Real-time Study Planner using JavaScript Timers (setInterval, setTimeout) and ES6 Modules (import / export).

2. Software / Tools Used

• HTML5, CSS3 and JavaScript (ES6 modules)

• Timers: setInterval(), clearInterval(), setTimeout(), clearTimeout()

• Visual Studio Code, Web Browser, local web server (modules need http://)

3. Program Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Study Planner - Timers & Modules</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <header>
      <h1>Study Planner</h1>
      <p>JavaScript timers (<code>setInterval</code>, <code>setTimeout</code>) with ES6 modules
(<code>import / export</code>)</p>
    </header>

    <!-- 1. Live clock : setInterval -->
    <section class="card clock">
      <div>
        <div class="label">Current Time</div>
        <div id="clockTime" class="big">--:--:--</div>
      </div>
      <div class="right">
        <div class="label">Today</div>
        <div id="clockDate" class="date">---</div>
      </div>
    </section>

    <div class="grid">
      <!-- 2. Focus timer : setInterval + clearInterval -->
      <section class="card">
        <h2>Focus Timer</h2>
        <label for="focusMinutes">Session length (minutes)</label>
        <input type="number" id="focusMinutes" min="1" max="180" value="25">
        <div id="focusDisplay" class="big timer">25:00</div>
        <div class="bar"><div id="focusBar" class="fill"></div></div>
        <div class="row">
          <button id="startBtn">Start</button>
          <button id="pauseBtn" class="grey">Pause</button>
          <button id="resetBtn" class="grey">Reset</button>
        </div>
        <p id="focusMsg" class="note">Set the length and press Start.</p>
      </section>

      <!-- 3. Reminder : setTimeout + clearTimeout -->
      <section class="card">
        <h2>Reminder (setTimeout)</h2>
        <label for="remText">Reminder text</label>
        <input type="text" id="remText" placeholder="e.g. Submit lab report">
```

```
        <label for="remSeconds">Remind me after (seconds)</label>
        <input type="number" id="remSeconds" min="1" max="3600" value="10">
        <div class="row">
          <button id="addRemBtn">Set Reminder</button>
        </div>
        <ul id="remList" class="list"></ul>
      </section>
    </div>

    <!-- 4. Exam countdown : setInterval -->
    <section class="card">
      <h2>Exam Countdown</h2>
      <div class="row start">
        <input type="text" id="examName" placeholder="Exam name" value="End Semester Exam">
        <input type="datetime-local" id="examDate">
        <button id="examBtn">Start Countdown</button>
      </div>
      <div id="examOut" class="units">
        <div><span id="dd">0</span><small>Days</small></div>
        <div><span id="hh">0</span><small>Hours</small></div>
        <div><span id="mm">0</span><small>Minutes</small></div>
        <div><span id="ss">0</span><small>Seconds</small></div>
      </div>
      <p id="examMsg" class="note">Pick a date and time to begin.</p>
    </section>

    <div id="toast" class="toast" role="status"></div>
    <footer>Vansh Mahant | PRN: 24070521038 | Section A | Batch A1</footer>
  </main>

  <!-- ES6 module entry point (must be served over http://) -->
  <script type="module" src="main.js"></script>
</body>
</html>
```

style.css

```
:root { --ink:#1f2937; --muted:#6b7280; --line:#e5e7eb; --brand:#0f766e; --brand-d:#115e59; --
bg:#f3f4f6; }
* { box-sizing: border-box; }
body { margin:0; font-family: "Segoe UI", Arial, sans-serif; background:var(--bg); color:var(--ink);
}
.app { max-width: 920px; margin: 0 auto; padding: 28px 16px 20px; }
header { text-align:center; margin-bottom: 20px; }
header h1 { margin:0 0 6px; color:var(--brand-d); }
header p { margin:0; color:var(--muted); font-size:14px; }
code { background:#e2e8f0; padding:1px 5px; border-radius:4px; }
.card { background:#fff; border:1px solid var(--line); border-radius:12px; padding:20px; margin-
bottom:18px; box-shadow:0 1px 3px rgba(0,0,0,.05); }
.card h2 { margin:0 0 12px; font-size:18px; color:var(--brand-d); }
.grid { display:grid; grid-template-columns:1fr 1fr; gap:18px; }
.grid .card { margin-bottom:0; }
.grid { margin-bottom:18px; }
.clock { display:flex; justify-content:space-between; align-items:center; background:var(--brand);
color:#fff; border:0; }
.clock .label { opacity:.8; font-size:13px; }
.clock .big { font-size:40px; font-weight:700; font-family:Consolas, monospace; }
.clock .right { text-align:right; }
.date { font-size:16px; }
label { display:block; font-size:13px; font-weight:600; margin:10px 0 4px; }
input { width:100%; padding:9px 10px; border:1px solid #cbd5e1; border-radius:6px; font-size:14px; }
.timer { text-align:center; font-size:54px; font-weight:700; font-family:Consolas, monospace;
margin:14px 0 8px; color:var(--brand-d); }
.bar { height:8px; background:var(--line); border-radius:4px; overflow:hidden; }
.fill { height:100%; width:0; background:var(--brand); transition:width .5s linear; }
.row { display:flex; gap:8px; margin-top:14px; flex-wrap:wrap; }
.row.start { align-items:center; margin-top:0; }
.row.start input { flex:1; min-width:150px; }
button { padding:9px 16px; background:var(--brand); color:#fff; border:0; border-radius:6px; font-
weight:600; cursor:pointer; }
button:hover { background:var(--brand-d); }
button.grey { background:#64748b; } button.grey:hover { background:#475569; }
button.mini { padding:3px 9px; font-size:12px; background:#dc2626; }
.note { color:var(--muted); font-size:13px; margin:10px 0 0; }
.list { list-style:none; padding:0; margin:14px 0 0; }
.list li { display:flex; justify-content:space-between; align-items:center; gap:8px; padding:8px 0;
border-top:1px solid var(--line); font-size:14px; }
.tag { font-size:11px; font-weight:700; padding:2px 8px; border-radius:10px; }
.tag.waiting { background:#fef3c7; color:#92400e; }
.tag.done { background:#d1fae5; color:#065f46; }
```

```
.tag.cancelled { background:#e5e7eb; color:#4b5563; }
.units { display:grid; grid-template-columns:repeat(4,1fr); gap:10px; margin-top:16px; text-
align:center; }
.units div { background:#f0fdfa; border:1px solid #99f6e4; border-radius:8px; padding:12px 4px; }
.units span { display:block; font-size:30px; font-weight:700; color:var(--brand-d); font-
family:Consolas, monospace; }
.units small { color:var(--muted); font-size:12px; }
.toast { position:fixed; right:20px; bottom:20px; background:#111827; color:#fff; padding:12px 18px;
border-radius:8px; font-size:14px; opacity:0; transform:translateY(10px); transition:.25s; pointer-
events:none; }
.toast.show { opacity:1; transform:none; }
footer { text-align:center; color:var(--muted); font-size:13px; margin-top:8px; }
@media (max-width:700px){ .grid{grid-template-columns:1fr;} .units{grid-template-
columns:repeat(2,1fr);} .clock{flex-direction:column;gap:8px;} }
```

main.js

```
// main.js - imports the modules and connects them to the page
import { startClock } from "./modules/clock.js";
import { FocusTimer, formatMMSS } from "./modules/focus.js";
import { addReminder, cancelReminder, allReminders } from "./modules/reminder.js";
import { startCountdown } from "./modules/countdown.js";

const $ = id => document.getElementById(id);

// ---- toast message (hidden again with setTimeout) ----
let toastTimer;
function toast(msg) {
  const t = $("toast");
  t.textContent = msg;
  t.classList.add("show");
  clearTimeout(toastTimer);
  toastTimer = setTimeout(() => t.classList.remove("show"), 4000);
}

// ---- 1. clock ----
startClock($("clockTime"), $("clockDate"));

// ---- 2. focus timer ----
const focus = new FocusTimer(
  (remaining, total) => {
    $("focusDisplay").textContent = formatMMSS(remaining);
    const done = total ? ((total - remaining) / total) * 100 : 0;
    $("focusBar").style.width = done + "%";
  },
  () => {
    $("focusMsg").textContent = "Session complete. Take a short break!";
    toast("Focus session finished");
  }
);
focus.setLength(Number($("focusMinutes").value) || 25);

$("focusMinutes").addEventListener("change", () => {
  const m = Math.min(180, Math.max(1, Number($("focusMinutes").value) || 25));
  $("focusMinutes").value = m;
  focus.setLength(m);
  $("focusMsg").textContent = `Timer set to ${m} minute(s).`;
});
$("startBtn").addEventListener("click", () => {
  focus.start();
  $("focusMsg").textContent = "Focus mode running...";
});
$("pauseBtn").addEventListener("click", () => {
  focus.stop();
  $("focusMsg").textContent = "Paused.";
});
$("resetBtn").addEventListener("click", () => {
  focus.reset();
  $("focusMsg").textContent = "Timer reset.";
});

// ---- 3. reminders ----
function renderReminders() {
  const ul = $("remList");
  ul.innerHTML = "";
  allReminders().forEach(r => {
    const li = document.createElement("li");
    li.innerHTML = `<span>${r.text} <small>(${r.seconds}s)</small></span>
                    <span class="tag ${r.status.toLowerCase()}">${r.status}</span>`;
    if (r.status === "Waiting") {
```

```
      const b = document.createElement("button");
      b.textContent = "Cancel";
      b.className = "mini";
      b.addEventListener("click", () => { cancelReminder(r.id); renderReminders(); });
      li.appendChild(b);
    }
    ul.appendChild(li);
  });
}

$("addRemBtn").addEventListener("click", () => {
  const text = $("remText").value.trim();
  const secs = Number($("remSeconds").value);
  if (!text || !(secs > 0)) { toast("Enter reminder text and seconds"); return; }
  addReminder(text, secs, r => { toast("Reminder: " + r.text); renderReminders(); });
  $("remText").value = "";
  renderReminders();
});

// ---- 4. exam countdown ----
let examTimer = null;
$("examBtn").addEventListener("click", () => {
  const value = $("examDate").value;
  if (!value) { $("examMsg").textContent = "Please choose a date and time."; return; }
  const target = new Date(value);
  if (target <= new Date()) { $("examMsg").textContent = "Choose a future date."; return; }
  clearInterval(examTimer);
  $("examMsg").textContent = `Counting down to ${$("examName").value || "exam"}...`;
  examTimer = startCountdown(
    target,
    p => { $("dd").textContent = p.days; $("hh").textContent = p.hours; $("mm").textContent =
p.minutes; $("ss").textContent = p.seconds; },
    () => { $("examMsg").textContent = "Time is up. Best of luck!"; toast("Countdown finished"); }
  );
});
```

modules/clock.js

```
// clock.js - live clock module (uses setInterval)

export function formatTime(d) {
  return d.toLocaleTimeString("en-GB");               // 24-hour HH:MM:SS
}

export function formatDate(d) {
  return d.toLocaleDateString("en-IN", { weekday: "long", day: "numeric", month: "long", year:
"numeric" });
}

// Starts the clock and returns the interval id so the caller can stop it
export function startClock(timeEl, dateEl) {
  function tick() {
    const now = new Date();
    timeEl.textContent = formatTime(now);
    dateEl.textContent = formatDate(now);
  }
  tick();
  return setInterval(tick, 1000);
}
```

modules/focus.js

```
// focus.js - focus (study) timer module: setInterval / clearInterval

export const pad = n => String(n).padStart(2, "0");

export function formatMMSS(totalSeconds) {
  return `${pad(Math.floor(totalSeconds / 60))}:${pad(totalSeconds % 60)}`;
}

export class FocusTimer {
  constructor(onTick, onDone) {
    this.onTick = onTick;     // called every second with (remaining, total)
    this.onDone = onDone;     // called when time is over
    this.total = 0;
    this.remaining = 0;
    this.id = null;
  }

  setLength(minutes) {
```

```
    this.stop();
    this.total = this.remaining = minutes * 60;
    this.onTick(this.remaining, this.total);
  }

  get running() { return this.id !== null; }

  start() {
    if (this.running || this.remaining <= 0) return;
    this.id = setInterval(() => {
      this.remaining--;
      this.onTick(this.remaining, this.total);
      if (this.remaining <= 0) {
        this.stop();
        this.onDone();
      }
    }, 1000);
  }

  stop() {                    // pause
    clearInterval(this.id);
    this.id = null;
  }

  reset() {
    this.stop();
    this.remaining = this.total;
    this.onTick(this.remaining, this.total);
  }
}
```

modules/reminder.js

```
// reminder.js - reminder module: setTimeout / clearTimeout

let nextId = 1;
const reminders = new Map();      // id -> { id, text, seconds, status, timer }

export function addReminder(text, seconds, onFire) {
  const id = nextId++;
  const r = { id, text, seconds, status: "Waiting" };
  r.timer = setTimeout(() => {
    r.status = "Done";
    onFire(r);
  }, seconds * 1000);
  reminders.set(id, r);
  return r;
}

export function cancelReminder(id) {
  const r = reminders.get(id);
  if (r && r.status === "Waiting") {
    clearTimeout(r.timer);
    r.status = "Cancelled";
  }
  return r;
}

export function allReminders() {
  return [...reminders.values()];
}
```

modules/countdown.js

```
// countdown.js - countdown to a future date (setInterval)

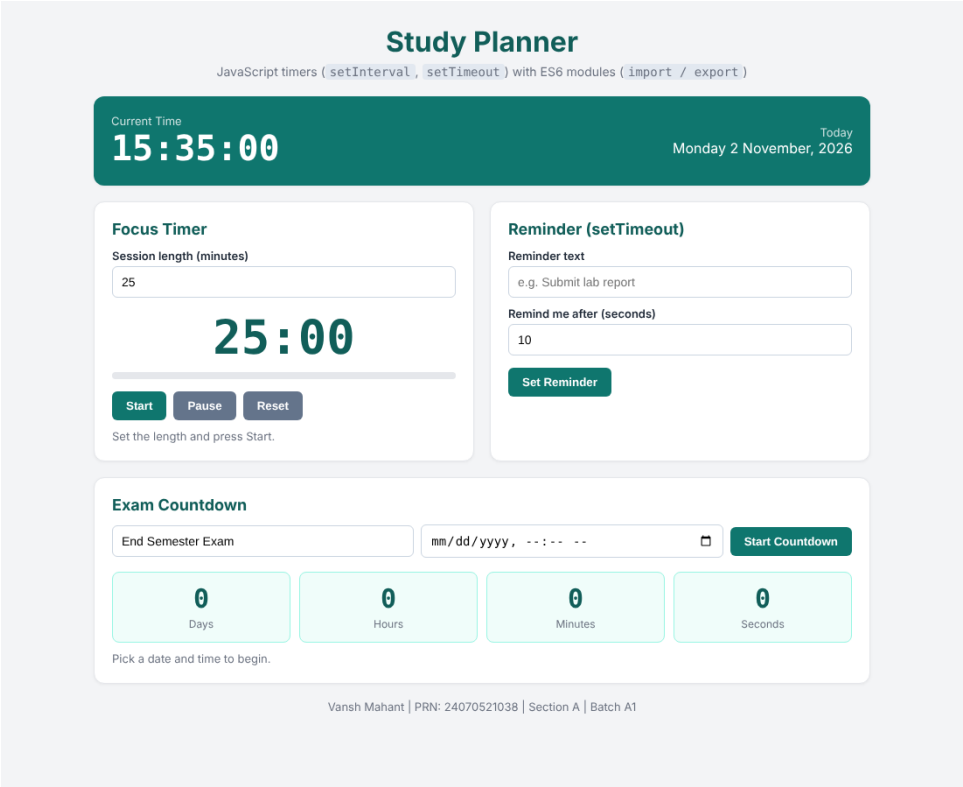
export function breakDown(ms) {
  const s = Math.max(0, Math.floor(ms / 1000));
  return {
    days: Math.floor(s / 86400),
    hours: Math.floor((s % 86400) / 3600),
    minutes: Math.floor((s % 3600) / 60),
    seconds: s % 60
  };
}

// Calls update() every second; returns the interval id
export function startCountdown(target, update, finished) {
  function tick() {
    const diff = target.getTime() - Date.now();
```

```
    update(breakDown(diff));
    if (diff <= 0) {
      clearInterval(id);
      finished();
    }
  }
  const id = setInterval(tick, 1000);
  tick();
  return id;
}
```

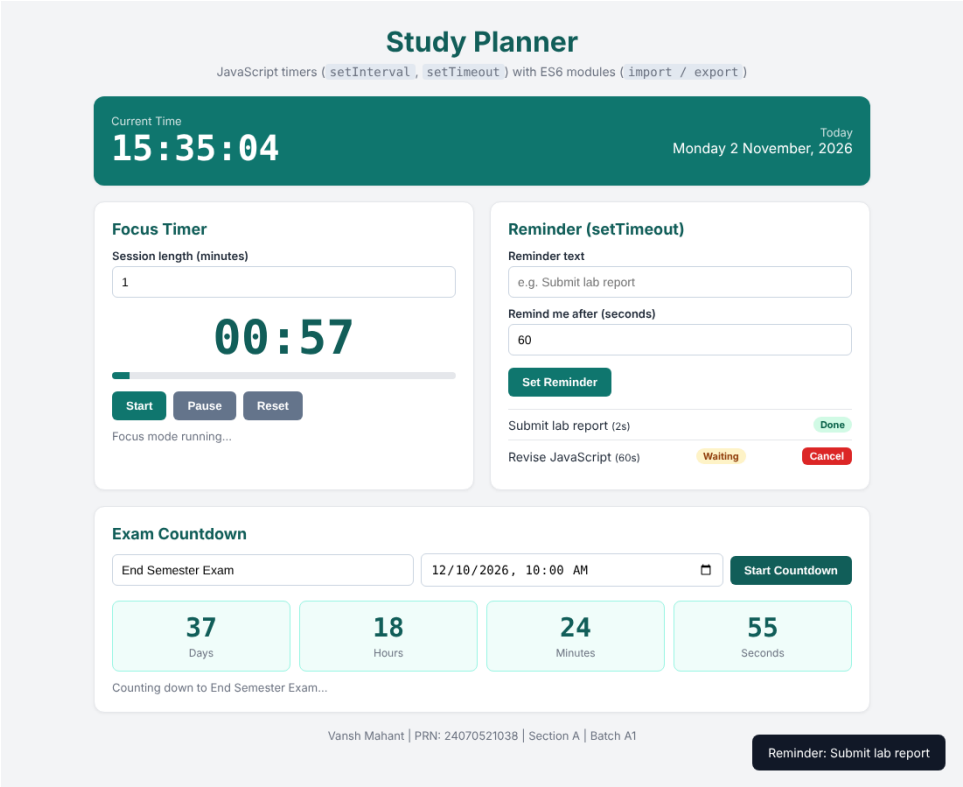
4. Output

Initial page: live clock updated every second with setInterval, focus timer set to 25:00.



PRN: 24070521038 | Name: Vansh Mahant | File: index.html

Focus timer running (00:57), reminder fired by setTimeout (status Done, toast shown), second reminder waiting with Cancel, and exam countdown running.



PRN: 24070521038 | Name: Vansh Mahant | File: index.html

5. Result / Conclusion

Timers were implemented with setInterval() for the live clock, focus timer and exam countdown, and with setTimeout() for reminders. clearInterval() and clearTimeout() were used to pause, reset and cancel. The code was split into ES6 modules (clock, focus, reminder, countdown) which are imported by main.js using import and export.